

Sequence Models

DL Specialization, Course 5 — detailed notes (Weeks 1–4)

RNNs / LSTMs · word embeddings · attention & seq2seq · transformers

1. Week 1 — Recurrent Neural Networks

1.1 Why sequence models & notation

Sequence data: speech, text, music, DNA, video. Types: sequence→sequence (translation, NER), sequence→one (sentiment), one→sequence (music gen). Notation: $x^{(t)}$ = t -th element of input; T_x, T_y = input/output lengths; $x^{(i)(t)}$ = element t of example i . Words are represented via a **vocabulary** + **one-hot** vectors (unknown words → <UNK>).

1.2 The RNN model

A standard net fails on sequences: inputs/outputs vary in length, and it doesn't share features across positions. An RNN processes one element at a time, passing an **activation** $a^{(t)}$ forward:

$$\begin{aligned}a^{(t)} &= g(W_{aa}a^{(t-1)} + W_{ax}x^{(t)} + b_a) \quad (\text{tanh/ReLU}), \\ \hat{y}^{(t)} &= g(W_{ya}a^{(t)} + b_y) \quad (\text{sigmoid/softmax}).\end{aligned}$$

W_{aa} (the “memory”), W_{ax} , W_{ya} are *shared across all time steps* — that's the feature-sharing. Simplified: stack into $W_a = [W_{aa} \mid W_{ax}]$, so $a^{(t)} = g(W_a[a^{(t-1)}, x^{(t)}] + b_a)$.

A basic (unidirectional) RNN only uses *past* context. “He said, Teddy Roosevelt...” vs. “He said, teddy bears...” — you need words *after* “Teddy” to disambiguate. Fixed by BiRNN (below).

1.3 Backpropagation through time

Same net unrolled in time; loss is summed over steps ($L = \sum_t L^{(t)}$ with cross-entropy per step). Gradients flow *backward through time* (right to left). Frameworks do it automatically.

1.4 Different types of RNNs

Many-to-many (equal $T_x=T_y$: NER), **many-to-one** (sentiment), **one-to-many** (music gen, feeding output back), **many-to-many unequal** (translation — *encoder* reads input, *decoder* emits output).

1.5 Language model & sampling

A **language model** gives $P(\text{sentence})$. Train on a corpus: tokenize, add <EOS>. At each step the RNN predicts the next word given previous ones; feed the *true* previous word during training.

$$P(y^{(1)}, y^{(2)}, y^{(3)}) = P(y^{(1)}) P(y^{(2)} \mid y^{(1)}) P(y^{(3)} \mid y^{(1)}, y^{(2)}).$$

Sampling novel sequences: at each step sample from the softmax (`np.random.choice`), feed the sampled word back in, until <EOS>. **Character-level** models handle any word (no <UNK>) but give longer sequences, worse long-range dependencies, and are costlier.

1.6 Vanishing gradients & GRU

Basic RNNs struggle with **long-range dependencies** (“The **cat**... **was** full” vs. “The **cats**... **were** full”): gradients vanish over many steps, so far-apart words can't influence each other. (Exploding gradients are easier — fix with **gradient clipping**.)

The **GRU** adds a *memory cell* c with gates:

$$\begin{aligned}\tilde{c}^{(t)} &= \tanh(W_c[\Gamma_r * c^{(t-1)}, x^{(t)}] + b_c), \\ \Gamma_u &= \sigma(W_u[c^{(t-1)}, x^{(t)}] + b_u) \quad (\text{update gate}), \\ \Gamma_r &= \sigma(W_r[c^{(t-1)}, x^{(t)}] + b_r) \quad (\text{relevance gate}), \\ c^{(t)} &= \Gamma_u * \tilde{c}^{(t)} + (1 - \Gamma_u) * c^{(t-1)}.\end{aligned}$$

When $\Gamma_u \approx 0$ the cell just *copies* $c^{(t-1)}$ across many steps ($c^{(t)} = c^{(t-1)}$) — carrying information far without vanishing.

1.7 LSTM

More general than GRU: separate **update**, **forget**, and **output** gates, and $c \neq a$:

$$\begin{aligned}\tilde{c}^{(t)} &= \tanh(W_c[a^{(t-1)}, x^{(t)}] + b_c), \\ \Gamma_u &= \sigma(\cdot) \text{ (update)}, \quad \Gamma_f = \sigma(\cdot) \text{ (forget)}, \quad \Gamma_o = \sigma(\cdot) \text{ (output)}, \\ c^{(t)} &= \Gamma_u * \tilde{c}^{(t)} + \Gamma_f * c^{(t-1)}, \quad a^{(t)} = \Gamma_o * \tanh(c^{(t)}).\end{aligned}$$

The c -line acts like a conveyor belt carrying memory forward. **GRU vs. LSTM:** GRU is simpler/faster (scales to bigger nets); LSTM is more powerful and the historical default. No universal winner.

1.8 Bidirectional & deep RNNs

BiRNN: one forward pass + one backward pass; $\hat{y}^{(t)}$ uses both $\vec{a}^{(t)}$ and $\overleftarrow{a}^{(t)}$ — so predictions use past *and* future context. BiLSTM is a common NLP default. Downside: needs the *whole* sequence before predicting (bad for live speech). **Deep RNNs:** stack a few recurrent layers (3 is already a lot / expensive).

2. Week 2 — Word Embeddings

2.1 Word representation

One-hot vectors treat every word as unrelated (all inner products 0) — no generalization (“orange juice” $\nrightarrow$ “apple juice”). **Word embeddings** give each word a dense vector (e.g. 300-D) capturing features (gender, royal, age, food...). Similar words get similar vectors; t-SNE shows related words clustering.

2.2 Using & properties

Transfer learning: learn embeddings from a huge unlabeled corpus (or download pretrained), then use them on a small-data task (NER, etc.); optionally fine-tune. Reduces input size (300-D vs. 10,000-D one-hot). **Analogies:** $e_{\text{man}} - e_{\text{woman}} \approx e_{\text{king}} - e_{\text{queen}}$ — so king $\rightarrow ?$ is $\arg \max_w \text{sim}(e_w, e_{\text{king}} - e_{\text{man}} + e_{\text{woman}})$, using **cosine similarity** $\frac{u^\top v}{\|u\| \|v\|}$.

2.3 Embedding matrix

$E \in \mathbb{R}^{300 \times 10000}$; one-hot o_j picks a column: $E o_j = e_j$ (in practice just slice the column — Keras **Embedding** layer). E is learned.

2.4 Word2Vec

Skip-gram: pick a context word c , predict a nearby target t ; model $P(t \mid c) = \text{softmax}(\theta_t^\top e_c)$.

The softmax denominator sums over the whole vocabulary (10k–1M) — **slow**. Fixes: hierarchical softmax, or negative sampling (below).

CBOW (the other Word2Vec variant) predicts the center word from context.

2.5 Negative sampling

Turn it into cheap binary classification. For a true (context, word) pair (target 1), draw k *negative* words (target 0):

$$P(y=1 \mid c, t) = \sigma(\theta_t^\top e_c).$$

Train only $k+1$ logistic classifiers per step instead of a full softmax. k : 5–20 (small data), 2–5 (large). Sample negatives by a $\propto f(w)^{3/4}$ heuristic (balances frequent vs. rare words).

2.6 GloVe

Simplest embedding algorithm. Let $X_{ij} = \#$ times word i appears in context of j . Minimize

$$\sum_{i,j} f(X_{ij}) (\theta_i^\top e_j + b_i + b'_j - \log X_{ij})^2,$$

f = weighting (0 when $X_{ij}=0$; down-weights very frequent words). θ and e are symmetric; average them for the final embedding.

You can't guarantee embedding axes are humanly interpretable (gender/age) — they're just useful directions.

2.7 Sentiment classification

With few labeled reviews, embeddings help. Simple model: average/sum word embeddings $\rightarrow$ softmax (ignores order — “lacking in good...” looks positive). Better: feed embeddings into a (many-to-one) RNN.

2.8 Debiasing word embeddings

Embeddings absorb societal bias (“man:programmer :: woman:homemaker”). Fix (Bolukbasi et al.):

1. **Identify** the bias direction (average $e_{he} - e_{she}$, etc.).
2. **Neutralize** non-definitional words (project off the bias direction).
3. **Equalize** pairs (grandmother/grandfather) to be symmetric about the neutral axis.

3. Week 3 — Sequence-to-Sequence & Attention

3.1 Basic seq2seq models

Encoder–decoder: an encoder RNN reads the input into a vector; a decoder RNN generates the output (machine translation). Same shape works for **image captioning** (CNN encoder $\rightarrow$ RNN decoder).

3.2 Picking the most likely sentence

Translation = a “conditional language model”: $P(y \mid x)$. Don't *sample* the output (random, sometimes bad); find the sequence maximizing $P(y \mid x)$. **Greedy** (best word each step) fails — picking the locally-best first word can force a worse sentence.

3.3 Beam search

Keep the top B partial sequences at each step (beam width B):

Step 1: keep the B best first words. Each next step: extend all B by every vocab word, keep the B highest-probability combinations. $B=1$ = greedy; larger B = better but slower.

Refinements: (1) sum *logs* of probabilities (avoid underflow); (2) **length normalization** — divide by T_y^α ($\alpha \approx 0.7$) so long sentences aren't unfairly penalized. Beam search is heuristic — not guaranteed optimal (unlike BFS/DFS).

3.4 Error analysis in beam search

Model produces $\hat{y}$, human gives y^* . Compare the model's own probabilities:

- $P(y^* | x) > P(\hat{y} | x)$: **beam search** is at fault (missed the better option).
- $P(y^* | x) \leq P(\hat{y} | x)$: the **RNN** is at fault (bad probabilities).

Count across many examples to decide where to invest.

3.5 BLEU score

Auto-evaluate translation against human references via **modified n-gram precision** (clip each n-gram count to its max in any reference). Combine $p_1 \dots p_4$: $\text{BP} \cdot \exp(\frac{1}{n} \sum p_n)$, with a **brevity penalty** BP that punishes too-short outputs.

3.6 Attention model

The encoder-decoder “memorize everything in one vector” breaks on long sentences. **Attention** lets the decoder look at a *weighted* part of the input at each output step (like a human translating a phrase at a time):

$$c^{(t)} = \sum_{t'} \alpha^{(t,t')} a^{(t')}, \quad \sum_{t'} \alpha^{(t,t')} = 1,$$
$$\alpha^{(t,t')} = \frac{\exp(e^{(t,t')})}{\sum_{t''} \exp(e^{(t,t'')})},$$

where $e^{(t,t')}$ comes from a small NN of the decoder state $s^{(t-1)}$ and encoder activation $a^{(t')}$. The weights α say “how much to attend to input position t' when generating output t .” Cost is quadratic ($T_x \cdot T_y$), but a big accuracy win on long sequences.

3.7 Speech & trigger word

Speech recognition: audio $\rightarrow$ transcript, on spectrograms; end-to-end DL removed hand-engineered phonemes. **CTC cost** allows outputs like `ttt_h_eee...` (collapse repeats not separated by “blank”), handling input $\gg$ output length. **Trigger word detection:** label 1 just after the trigger word (0 elsewhere); imbalanced, so output a few 1s per trigger.

4. Week 4 — Transformer Network

4.1 Intuition

RNNs process sequentially (slow, hard gradient flow); attention alone lets you process a whole sequence *in parallel* and directly connect any two positions. **Transformer** = **self-attention** + **multi-head attention**, **no recurrence**. Foundation of modern NLP (BERT, GPT, ...).

4.2 Self-attention

Each word gets three vectors from learned matrices: **query** $q = W^Q x$, **key** $k = W^K x$, **value** $v = W^V x$. For word i , attend to all words by matching its query against every key:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Read it per word: $A(q, K, V) = \sum_j \frac{\exp(q \cdot k_j)}{\sum_{j'} \exp(q \cdot k_{j'})} v_j$ — a weighted sum of *values*, weights = how relevant each other word is. $\sqrt{d_k}$ scales the dot products for stable gradients. Unlike RNNs, all positions computed at once, and any word can directly attend to any other.

4.3 Multi-head attention

Run self-attention h times in parallel with different learned W^Q, W^K, W^V (each a “head” asking a different question — who? when? what?), concatenate, and project:

$$\text{MultiHead} = \text{concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V).$$

4.4 Transformer network

Encoder (repeated $N \times$): multi-head self-attention $\rightarrow$ feed-forward, each with residual + layer-norm. **Decoder** (repeated $N \times$): *masked* multi-head self-attention (can’t peek ahead) $\rightarrow$ multi-head attention over the *encoder* output (Q from decoder, K,V from encoder) $\rightarrow$ feed-forward, then a final linear+softmax.

Positional encoding: since there’s no recurrence, add sinusoidal position vectors so the model knows word order:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right).$$

Also uses residual connections, layer normalization, and (in training) the decoder mask. Applications: named-entity recognition, question answering, and the LLMs that followed.

Big picture of Course 5 (and the whole specialization): Week 1 builds recurrence — RNNs share weights across time; GRU/LSTM gates carry memory far enough to fix vanishing gradients; BiRNNs add future context. Week 2 represents *words* as dense embeddings (Word2Vec / negative sampling / GloVe) that generalize and encode analogies. Week 3 does *sequence-to-sequence* (encoder–decoder) with beam search and, crucially, **attention** — look at the relevant input parts per output step. Week 4 drops recurrence entirely: the **Transformer** is pure self-attention + multi-head attention + positional encoding, processed in parallel — the architecture behind today’s large language models.